

Practico 03 – Autoreferencia y Teorema de la Recursión

Computabilidad y Complejidad (Teoría de la Computación II)
FCEFYQyN – Universidad Nacional de Río Cuarto
Primer Cuatrimestre de 2026

Resolución completa del Práctico 03. Apoyada en el Capítulo 6 del libro de M. Sipser, *Introduction to the Theory of Computation*, y en las láminas teóricas de la cátedra ([autoref.pdf](#)).

Índice

Introducción	1
1. Ejercicio 1: lectura del Capítulo 6	2
2. Ejercicio 2: la función <code>Self</code> en Python	2
3. Ejercicio 3: dos MT que se imprimen mutuamente	4
4. Ejercicio 4: cardinalidad y oráculo A_{TM}	5
5. Ejercicio 5: clase <code>Persona</code> e introspección	6
6. Ejercicio 6: creación dinámica de una clase	8
Conclusiones	10

Introducción

El presente trabajo resuelve la totalidad de los ejercicios del Práctico 03 de la asignatura, centrado en la *autoreferencia* y el *Teorema de la Recursión*. Estas herramientas permiten que las Máquinas de Turing obtengan su propia descripción y operen con ella, lo cual da lugar a una nueva demostración de la indecidibilidad de A_{TM} , a propiedades llamativas como la no-reconocibilidad de MIN_{TM} , y a la noción de *introspección* en lenguajes de programación reales. La guía teórica utilizada es:

- el Capítulo 6 del libro de Sipser ([chapter-06.pdf](#)), que cubre el Teorema de la Recursión, sus aplicaciones (máquinas mínimas, teorema del punto fijo) y las nociones de *oracle Turing machine* y de *Turing reducibility*;
- las láminas de la cátedra ([autoref.pdf](#)) de Pablo Castro, que sintetizan la construcción de la máquina `SELF`, su versión recursiva general, el lenguaje MIN_{TM} y la noción de oráculo.

Convenciones de notación. A lo largo del documento adoptamos las siguientes convenciones, en línea con Sipser:

- Σ es un alfabeto y Σ^* el conjunto de cadenas finitas sobre Σ .
- $\langle M \rangle$ denota una codificación (descripción) de la Máquina de Turing M , y $\langle M, w \rangle$ codifica el par (M, w) .
- $\mathcal{L}(M)$ es el lenguaje reconocido por M .
- M^O denota una MT M con un oráculo para el lenguaje O (Definición 6.18 de Sipser).
- $A \leq_T B$ significa que A es decidable relativo a B , es decir, existe una MT con oráculo M^B que decide A (Definición 6.20 de Sipser).
- “Decidable” = Turing-decible; “reconocible” = Turing-reconocible.

Lemas de referencia. La herramienta básica de toda esta sección es el siguiente lema, que se usa repetidamente:

Lema 0.1 (Sipser 6.1). *Existe una función computable $q : \Sigma^* \rightarrow \Sigma^*$ tal que, para toda $w \in \Sigma^*$, $q(w)$ es la descripción $\langle P_w \rangle$ de una Máquina de Turing P_w que en cualquier entrada borra la cinta, escribe w y se detiene.*

A partir de él, Sipser construye la máquina SELF y demuestra el Teorema 6.3 (Recursión), que es la versión “parametrizada” de SELF.

1. Ejercicio 1: lectura del Capítulo 6

El primer ítem del práctico consiste en *leer el Capítulo 6 del libro de Sipser*. Se asume realizado como prerrequisito teórico para el resto de los ejercicios. Las nociones que se utilizan a partir del Ejercicio 2 incluyen, entre otras:

- el Lema 0.1 (Sipser 6.1) y la construcción de la máquina SELF que imprime su propia descripción;
- el Teorema de la Recursión (Sipser 6.3) y su aplicación a la indecibilidad de A_{TM} (Teorema 6.5);
- la no reconocibilidad de MIN_{TM} (Teorema 6.7);
- la versión de punto fijo de la recursión (Teorema 6.8);
- la noción formal de *oráculo* y de *Turing reducibility* (Definiciones 6.18 y 6.20);
- el resultado de cardinalidad: el conjunto de lenguajes es incontable (Corolario 4.18) y, por lo tanto, existen lenguajes no reconocibles aún por MT con oráculo (Problema 6.19).

Cada ejercicio que sigue cita explícitamente las definiciones y resultados del capítulo que invoca.

2. Ejercicio 2: la función Self en Python

Enunciado

Escribir en Python la función SELF que se reproduce a sí misma.

Idea y conexión con la teoría

La máquina SELF de Sipser está construida en dos partes A y B (páginas 246–248). Por el Lema 0.1 tomamos $A = P_{\langle B \rangle}$, la máquina que imprime $\langle B \rangle$ en la cinta. Después, B lee de la cinta $\langle B \rangle$, aplica q para reconstruir $\langle A \rangle$, y combina ambas mitades para imprimir $\langle AB \rangle = \langle \text{SELF} \rangle$.

En un lenguaje de programación real esta construcción se traduce en un *quine*: un programa que produce una copia exacta de su código fuente.

Implementación en Python

```
# self.py - Quine clasico estilo Sipser
# Parte A: la sentencia print(...) que imprime la cadena B.
# Parte B: la cadena B que describe el programa completo, con un
# marcador {!r} que se reemplaza por la propia B citada.
B = 'B = {!r}\nprint(B.format(B))\n'
print(B.format(B))
```

Correspondencia con SELF. La siguiente tabla muestra cómo se corresponden las componentes:

Sipser (SELF)	Python (self.py)
B (cadena base)	literal de la variable <code>B</code>
$A = P_{\langle B \rangle}$	<code>print(B.format(B))</code>
q (computa P_w)	especificador <code>{!r}</code> de <code>format</code>
salida $\langle AB \rangle$	texto impreso por <code>print</code>

Verificación. La corrección del quine se comprueba ejecutando

```
diff <(python3 self.py) self.py
```

que no debe producir salida. Esto es porque `repr(B)` genera el literal exacto de `B` que aparece en el código, y la cadena `B` describe las dos líneas del programa.

Variante mediante introspección

Una versión más cercana al enunciado teórico “una MT obtiene su propia descripción” usa el módulo `inspect`:

```
import inspect

def self_program():
    """Imprime su propio código fuente."""
    print(inspect.getsource(self_program))

if __name__ == "__main__":
    self_program()
```

Aquí `inspect.getsource` cumple el rol de la operación “obtener $\langle M \rangle$ ” que el Teorema 6.3 garantiza para una MT. La línea `self_program()` es la *ejecución* de esa descripción.

Conclusión

La existencia del quine en Python es una evidencia práctica del Teorema de la Recursión: cualquier lenguaje de programación Turing-completo permite escribir un programa que produce una copia exacta de sí mismo. La construcción de Sipser para SELF se traduce directamente en el patrón $A + B$ usando `format` y `repr`.

3. Ejercicio 3: dos MT que se imprimen mutuamente

Enunciado

Describir dos Máquinas de Turing M y N tales que, comenzando en cualquier input, M imprima $\langle N \rangle$ y N imprima $\langle M \rangle$.

Observacion. El enunciado del práctico (`practica03.pdf`) tiene una errata menor: pide “ M imprime $\langle M \rangle$ y N imprime $\langle M \rangle$ ”, lo cual es trivial (dos copias de SELF resuelven el caso). Tanto el Problema 6.6 de Sipser como las láminas de cátedra (`autoref.pdf`) piden imprimirse *mutuamente*, y esa es la versión interesante que resolvemos.

Estrategia

La técnica es la misma que la de SELF, agregando un *paso de transformación* al final. Por el Lema 0.1, dada cualquier cadena w , sabemos construir computacionalmente $\langle P_w \rangle$. En particular, podemos enviar

$$\langle N \rangle \mapsto q(\langle N \rangle) = \langle P_{\langle N \rangle} \rangle.$$

Si definimos $M = P_{\langle N \rangle}$, entonces $q(\langle N \rangle)$ es *exactamente* $\langle M \rangle$. Esto convierte “imprimir $\langle M \rangle$ ” en “imprimir $q(\langle N \rangle)$ ”, una tarea computable a partir de $\langle N \rangle$.

Construcción

Definición de M . M es la máquina del Lema 0.1 aplicada a $\langle N \rangle$:

M = “**Con entrada x :**

1. Ignorar x .
2. Borrar la cinta y escribir la cadena $\langle N \rangle$ (cableada en la descripción de M).
3. Detenerse.”

Es decir, $M = P_{\langle N \rangle}$, donde $\langle N \rangle$ aparece como constante en la descripción de M .

Definición de N . N se construye al estilo SELF, en partes A_N y B_N :

N :

1. **Parte A_N .** Es $P_{\langle B_N \rangle}$: ejecutarla deja $\langle B_N \rangle$ en la cinta.
2. **Parte B_N .** En entrada $\langle B_N \rangle$:
 - a) computar $q(\langle B_N \rangle) = \langle A_N \rangle$;
 - b) combinar A_N y B_N para obtener $\langle A_N B_N \rangle = \langle N \rangle$;
 - c) computar $q(\langle N \rangle) = \langle M \rangle$;
 - d) borrar la cinta, escribir $\langle M \rangle$ y detenerse.

Por qué no hay circularidad

A primera vista parecería que M y N se definen mutuamente y la construcción es circular. No lo es:

- N calcula $\langle N \rangle$ en tiempo de ejecución (al estilo SELF, lo reconstruye desde la cinta), y recién luego produce $\langle M \rangle$ aplicando q .
- M , en cambio, tiene $\langle N \rangle$ cableado como constante; pero $\langle N \rangle$ ya está fijo una vez construida la máquina N .

Demostración alternativa por el Teorema de la Recursión

Teorema 3.1 (Sipser 6.3). *Sea T una MT que computa $t : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. Entonces existe una MT R que computa $r : \Sigma^* \rightarrow \Sigma^*$ con $r(w) = t(\langle R \rangle, w)$ para todo w .*

Aplicando el Teorema con la transformación $t(\langle R \rangle, w) = q(\langle R \rangle)$ obtenemos directamente una MT N que en cualquier entrada w produce $q(\langle N \rangle) = \langle M \rangle$. La máquina M se obtiene como $P_{\langle N \rangle}$.

Correctitud

Demostración. ■ Por construcción de P_w (Lema 0.1), M ignora su entrada y deja $\langle N \rangle$ en la cinta. Luego M imprime $\langle N \rangle$.

■ Por la construcción de SELF (Lema 0.1 aplicado a B_N) N reconstruye su propia descripción $\langle N \rangle$ en la cinta. El único paso adicional es aplicar q una vez más para obtener $q(\langle N \rangle) = \langle M \rangle$. Luego N imprime $\langle M \rangle$.

□

Conclusión

N usa la receta de SELF y aplica q una vez más para imprimir $\langle M \rangle$; $M = P_{\langle N \rangle}$.

4. Ejercicio 4: cardinalidad y oráculo A_{TM}

Enunciado

Usar la propiedad de que el conjunto de lenguajes es incontable para demostrar que existen lenguajes que no pueden ser reconocidos con una máquina con un oráculo A_{TM} .

Idea

Argumento clásico de cardinalidad (Cantor):

- El conjunto de lenguajes sobre Σ es *incontable* (Corolario 4.18 de Sipser).
- El conjunto de MT con oráculo A_{TM} es *contable*: cada una se describe con una cadena finita.
- La imagen de un conjunto contable bajo cualquier función es contable; en particular, la familia de lenguajes reconocidos por MT con oráculo A_{TM} es contable.
- Una función no puede ser sobreyectiva de un conjunto contable a uno incontable.

Por lo tanto, existen lenguajes que ninguna MT con oráculo A_{TM} reconoce.

Recordatorios

Teorema 4.1 (Sipser, Corolario 4.18). *El conjunto $\mathcal{P}(\Sigma^*)$ de lenguajes sobre Σ es incontable.*

Definición 4.2 (Sipser 6.18). Un *oráculo* para el lenguaje B es un dispositivo externo capaz de responder, en un único paso, si una cadena w pertenece o no a B . Una *MT con oráculo*, notada M^B , es una MT modificada con la capacidad de hacer consultas al oráculo. La descripción $\langle M^B \rangle$ de una MT con oráculo es finita (no incluye al oráculo en sí; solo contiene los estados de consulta).

Demostración

Proposición 4.3. *Existe un lenguaje $L \subseteq \Sigma^*$ que no es reconocido por ninguna MT con oráculo A_{TM} .*

Demostración. Sea $\mathcal{M}_{A_{\text{TM}}}$ la familia de MT con oráculo A_{TM} . La asignación

$$M^{A_{\text{TM}}} \mapsto \langle M^{A_{\text{TM}}} \rangle \in \Sigma^*$$

es una inyección, ya que distintas MT producen distintas descripciones. Como Σ^* es contable, $\mathcal{M}_{A_{\text{TM}}}$ también lo es.

Definimos

$$\mathcal{R}_{A_{\text{TM}}} = \{ \mathcal{L}(M) : M \in \mathcal{M}_{A_{\text{TM}}} \}.$$

La imagen de un conjunto contable bajo cualquier función es contable, por lo que $\mathcal{R}_{A_{\text{TM}}}$ es contable.

Por el Teorema 4.1, $\mathcal{P}(\Sigma^*)$ es incontable. Luego

$$|\mathcal{R}_{A_{\text{TM}}}| \leq \aleph_0 < 2^{\aleph_0} = |\mathcal{P}(\Sigma^*)|,$$

y por lo tanto $\mathcal{R}_{A_{\text{TM}}} \subsetneq \mathcal{P}(\Sigma^*)$. Existe $L \in \mathcal{P}(\Sigma^*) \setminus \mathcal{R}_{A_{\text{TM}}}$; ese L no es reconocido por ninguna MT con oráculo A_{TM} . $\square$

Interpretación

Observación. Una MT con oráculo A_{TM} es muy poderosa: decide A_{TM} por consulta directa, y por construcción adicional decide E_{TM} , $HALT_{\text{TM}}$, EQ_{TM} , etc. (Ejemplo 6.19 y ejercicios de Sipser). Sin embargo, el argumento de cardinalidad muestra que su poder es “solamente contable”: se quedan incontables lenguajes fuera. Este resultado es *robusto* bajo el cambio de oráculo: para cualquier lenguaje fijo B existen lenguajes no reconocibles por MT con oráculo B .

Conclusión

$$\exists L \subseteq \Sigma^* \text{ tal que } L \notin \{ \mathcal{L}(M^{A_{\text{TM}}}) : M \text{ es MT} \}.$$

5. Ejercicio 5: clase Persona e introspección

Enunciado

Escribir una clase `Persona` en Python con los atributos clásicos. Y hacer una función que con introspección:

- imprima la lista de atributos de la clase;
- imprima el código de alguno de los métodos.

Contexto teórico

El Teorema de la Recursión dice que toda MT puede obtener su propia descripción $\langle M \rangle$ y operar con ella. En un lenguaje de programación moderno, esto se concreta como *introspección*: posibilidad de consultar, en tiempo de ejecución, el código y la estructura de los propios objetos. Python lo expone mediante el módulo estándar `inspect`.

Clase Persona

```
# persona.py
class Persona:
    """Representa una persona con datos personales clasicos."""

    especie = "Homo sapiens" # atributo de clase

    def __init__(self, nombre: str, edad: int, dni: str, email: str):
        self.nombre = nombre
        self.edad = edad
        self.dni = dni
        self.email = email

    def saludar(self) -> str:
        return f"Hola, soy {self.nombre} y tengo {self.edad} anios."

    def es_mayor_de_edad(self) -> bool:
        return self.edad >= 18

    def __str__(self) -> str:
        return f"Persona(nombre={self.nombre!r}, edad={self.edad}, dni={self.dni!r})"
```

Los atributos `nombre`, `edad`, `dni`, `email` son de *instancia* y se crean en `__init__`; `especie` es un atributo de *clase*.

Función de introspección

```
import inspect

def inspeccionar(cls, metodo: str | None = None) -> None:
    """Lista atributos y muestra el código de un método de la clase."""
    print(f"--- Inspeccion de la clase {cls.__name__} ---")

    miembros = inspect.getmembers(cls)
    atributos = [n for n, _ in miembros if not n.startswith("__")]
    print("Atributos y metodos publicos:")
    for nombre in atributos:
        print(f" - {nombre}")

    print("\nClasificacion:")
    for nombre in atributos:
        valor = getattr(cls, nombre)
        tipo = "metodo" if inspect.isfunction(valor) else "atributo de clase"
        print(f" - {nombre} ({tipo})")

    if metodo is None:
        metodos = [n for n, v in miembros
                    if inspect.isfunction(v) and not n.startswith("__")]
```

```

        if not metodos:
            print("\n(No hay metodos publicos para mostrar.)")
            return
        metodo = metodos[0]

        print(f"\nCodigo del metodo '{metodo}':")
        print(inspect.getsource(getattr(cls, metodo)))

if __name__ == "__main__":
    inspeccionar(Persona)
    inspeccionar(Persona, "es_mayor_de_edad")

```

Explicación.

- `inspect.getmembers(cls)` devuelve todos los miembros de la clase; filtramos los `dunder`.
- `inspect.isfunction` permite separar métodos de atributos de clase.
- `inspect.getsource` reconstruye, a partir del archivo fuente, el texto exacto del método. Es el equivalente real al “obtener $\langle M \rangle$ ” del Teorema de la Recursión.

Observaciones

Los atributos de instancia no aparecen al inspeccionar la clase, porque existen solamente después de instanciar. Para verlos:

```
p = Persona('.^da", 36, "0", .^da@x.org"); print(vars(p)).
```

Conclusión

`inspect` provee de fábrica todas las herramientas necesarias para la introspección pedida. Su existencia es la traducción concreta, en Python, de la libertad de una MT para acceder a su propia descripción que el Teorema de la Recursión garantiza.

6. Ejercicio 6: creación dinámica de una clase

Enunciado

Usando introspección, hacer una función que cree una clase dinámicamente.

Idea

Python expone `type(nombre, bases, atributos)` como constructor universal de clases. Llamándolo en tiempo de ejecución con tres argumentos crea una clase nueva, igual a la que produciría el azúcar sintáctico `class`. Combinado con `inspect`, podemos construir clases “a la carta” a partir de descripciones, en una analogía directa con la construcción q del Lema 0.1.

Código

```

# clase_dinamica.py
import inspect
from typing import Callable, Iterable

def crear_clase(

```



```

nombre: str,
campos: Iterable[str],
metodos: dict[str, Callable] | None = None,
bases: tuple[type, ...] = (object,),
):
    """Crea y devuelve una nueva clase usando type(...) y luego la inspecciona."""
    metodos = metodos or {}
    campos = list(campos)

    def __init__(self, *args, **kwargs):
        if len(args) > len(campos):
            raise TypeError(
                f"{nombre}.__init__() recibe a lo sumo {len(campos)} argumentos"
            )
        for c, v in zip(campos, args):
            setattr(self, c, v)
        for c, v in kwargs.items():
            if c not in campos:
                raise TypeError(f"campo desconocido para {nombre}: {c!r}")
            setattr(self, c, v)

    def __repr__(self):
        partes = ", ".join(f"{c}={getattr(self, c, None)!r}" for c in campos)
        return f"{nombre}({partes})"

    atributos = {
        "__init__": __init__,
        "__repr__": __repr__,
        "__doc__": f"Clase generada dinamicamente con campos: {campos}",
        **metodos,
    }
    cls = type(nombre, bases, atributos)

    print(f"[ok] Clase creada: {cls.__name__}")
    print(f"    Bases      : {tuple(b.__name__ for b in cls.__bases__)}")
    metodos_pub = [n for n, _ in inspect.getmembers(cls, inspect.isfunction)
                    if not n.startswith("__")]
    print(f"    Metodos    : {metodos_pub or '(ninguno)'}")
    print(f"    Firma init : {inspect.signature(cls.__init__)}")
    return cls

if __name__ == "__main__":
    Punto = crear_clase(
        "Punto",
        campos=["x", "y"],
        metodos={
            "norma": lambda self: (self.x ** 2 + self.y ** 2) ** 0.5,
            "trasladar": lambda self, dx, dy: Punto(self.x + dx, self.y + dy),
        },
    )
    p = Punto(3, 4)
    print("Instancia      :", p)
    print("norma          :", p.norma())
    print("trasladar       :", p.trasladar(1, 1))

```

Conexión con la teoría

Observacion. La función `crear_clase` es la versión programable de la construcción del Lema 0.1: así como q toma una cadena w y produce $\langle P_w \rangle$, nuestra función toma una descripción (nombre + bases + diccionario de atributos) y produce una clase. `type` juega el rol de la

transformación q , e `inspect` permite verificar el resultado.

Conclusión

`type` y `inspect` son suficientes para crear clases en tiempo de ejecución y validar su estructura, completando el ciclo de *metaprogramación*. Esto cierra la analogía entre el Teorema de la Recursión (capacidad de operar sobre la propia descripción) y las facilidades estándar de Python para inspección y construcción dinámica de objetos.

Conclusiones

A lo largo del Práctico 03 hemos puesto en práctica el *Teorema de la Recursión* y sus aplicaciones:

- Los Ejercicios 2 y 3 construyen autoreferencia: una MT (y su análogo en Python) que se imprime a sí misma, y un par de MT que se imprimen *mutuamente*. Ambos siguen la receta del Lema 0.1.
- El Ejercicio 4 muestra que, aun aumentándole a una MT el poder de un oráculo A_{TM} , sigue existiendo una cantidad incontable de lenguajes que escapa a su alcance. El argumento es de cardinalidad (Cantor) y es robusto bajo cambios de oráculo.
- Los Ejercicios 5 y 6 traen el Teorema de la Recursión al terreno práctico: el módulo `inspect` y la función `type` de Python son la versión operativa de “obtener la propia descripción” y “construir nuevas máquinas a partir de descripciones”.

En conjunto, los resultados consolidan la imagen de la autoreferencia como una herramienta central para entender los límites y las capacidades del modelo de cómputo, y como un puente concreto entre la teoría y la programación real.